

J.P. DAWER INSTITUTE OF INFORMATION AND
COMMUNICATION TECHNOLOGY

Academic Year: 2026 - 27

304 – Open Source Web Development

Practical Assignment - 2

➤ Submitted by:

Name: Thakkar Disha Vipulbhai

Class: ICT-3

Rollno.: 54

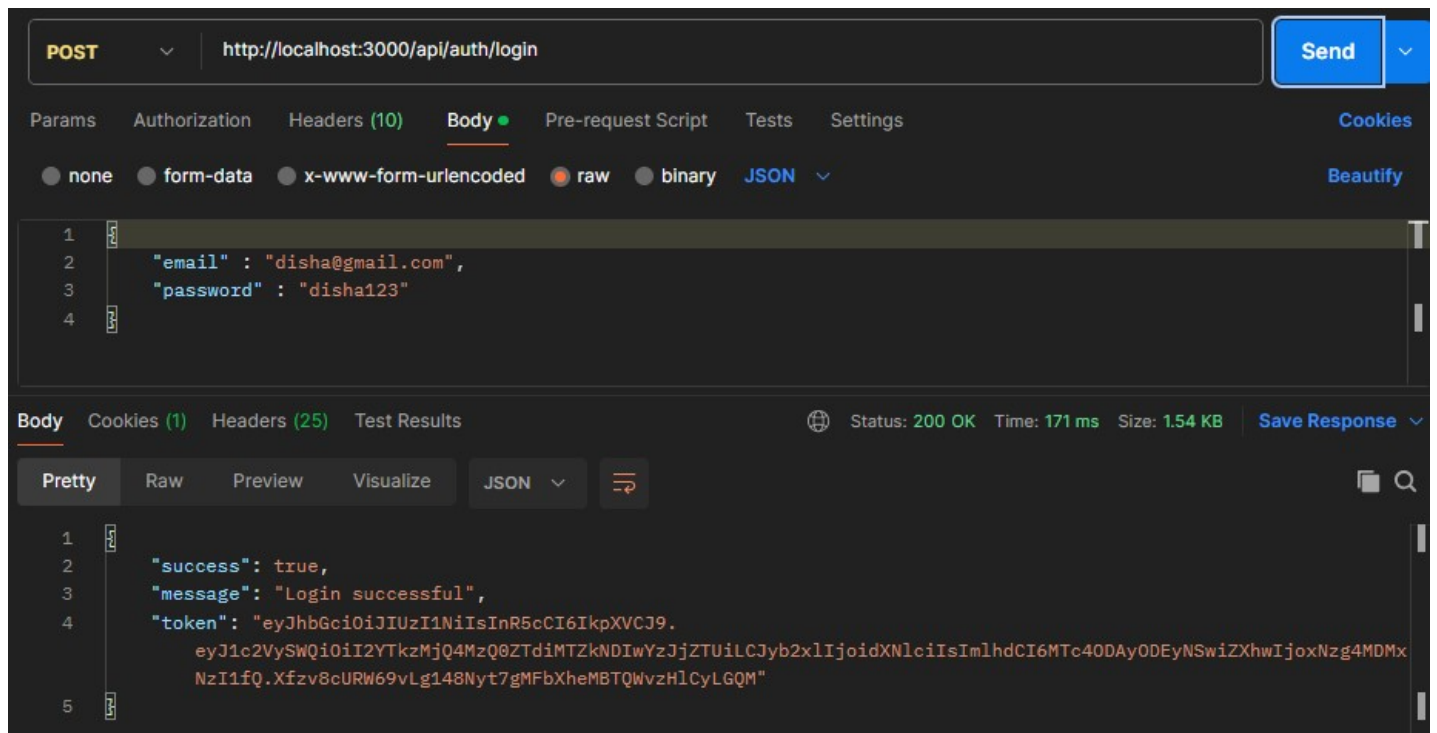
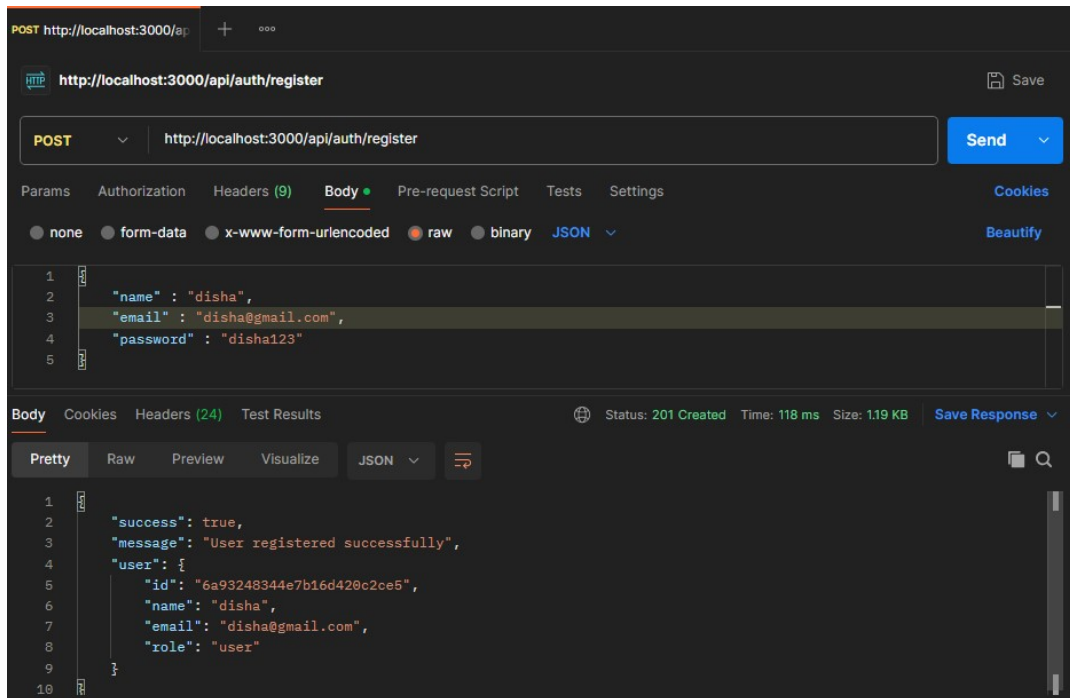
➤ Submitted to:

Prof. Rupal Panchal

Que.1 : Blog Management REST API with Authentication (Node.js, Express, MongoDB, EJS)

Output :

- **User Authentication**



GET ▼ http://localhost:3000/api/auth/profile Send ▼

Params **Authorization •** Headers (9) Body Pre-request Script Tests Settings Cookies

Type Bearer T... ▼ Token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

The authorization header will be automatically generated when you send the request.
[Learn more about authorization](#) ➤

Store your secrets with end-to-end encryption locally using Vault. Store in Vault

Body Cookies (1) Headers (21) Test Results 🌐 Status: 200 OK Time: 15 ms Size: 1.15 KB Save Response ▼

Pretty Raw Preview Visualize JSON ▼

```
1 {
2   "success": true,
3   "user": {
4     "_id": "6a93248344e7b16d420c2ce5",
5     "name": "disha",
6     "email": "disha@gmail.com",
7     "role": "user",
8     "createdAt": "2026-08-29T18:27:15.132Z",
9     "updatedAt": "2026-08-29T18:27:15.132Z",
10    "v": 0
11  }
```

- **Blog Post CRUD API (protected routes, linked to the logged-in author)**

POST ▼ http://localhost:3000/api/posts Send ▼

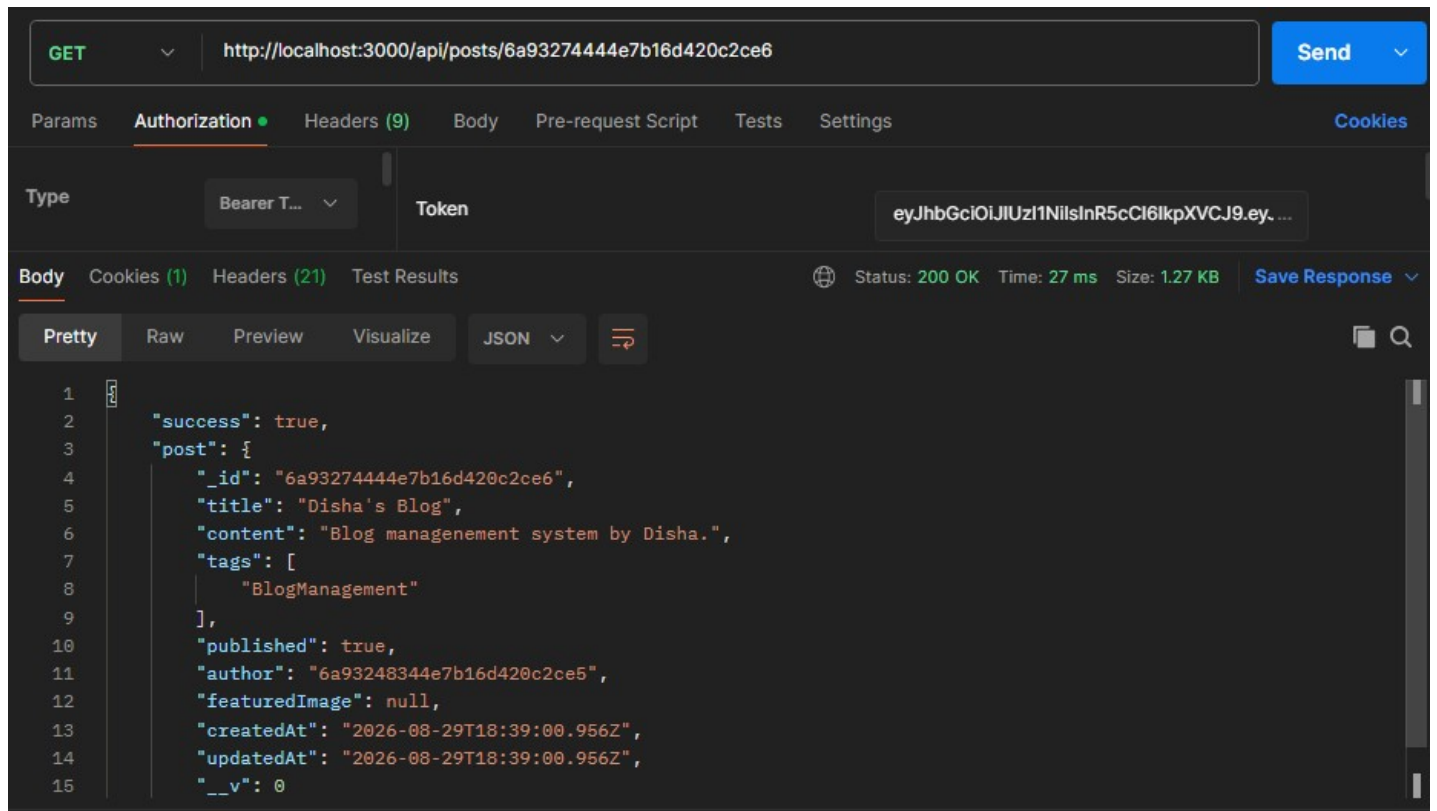
Params **Authorization •** Headers (11) Body • Pre-request Script Tests Settings Cookies

Type Bearer T... ▼ Token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

Body Cookies (1) Headers (21) Test Results 🌐 Status: 201 Created Time: 24 ms Size: 1.31 KB Save Response ▼

Pretty Raw Preview Visualize JSON ▼

```
1 {
2   "success": true,
3   "message": "Post created successfully",
4   "post": {
5     "title": "Disha's Blog",
6     "content": "Blog managenement system by Disha.",
7     "tags": [
8       "BlogManagement"
9     ],
10    "published": true,
11    "author": "6a93248344e7b16d420c2ce5",
12    "featuredImage": null,
13    "_id": "6a93274444e7b16d420c2ce6",
14    "createdAt": "2026-08-29T18:39:00.956Z",
15    "updatedAt": "2026-08-29T18:39:00.956Z",
16  }
```



- EJS View

Login

Email

Password

Login

My Dashboard

Welcome, disha!

Disha's Blog



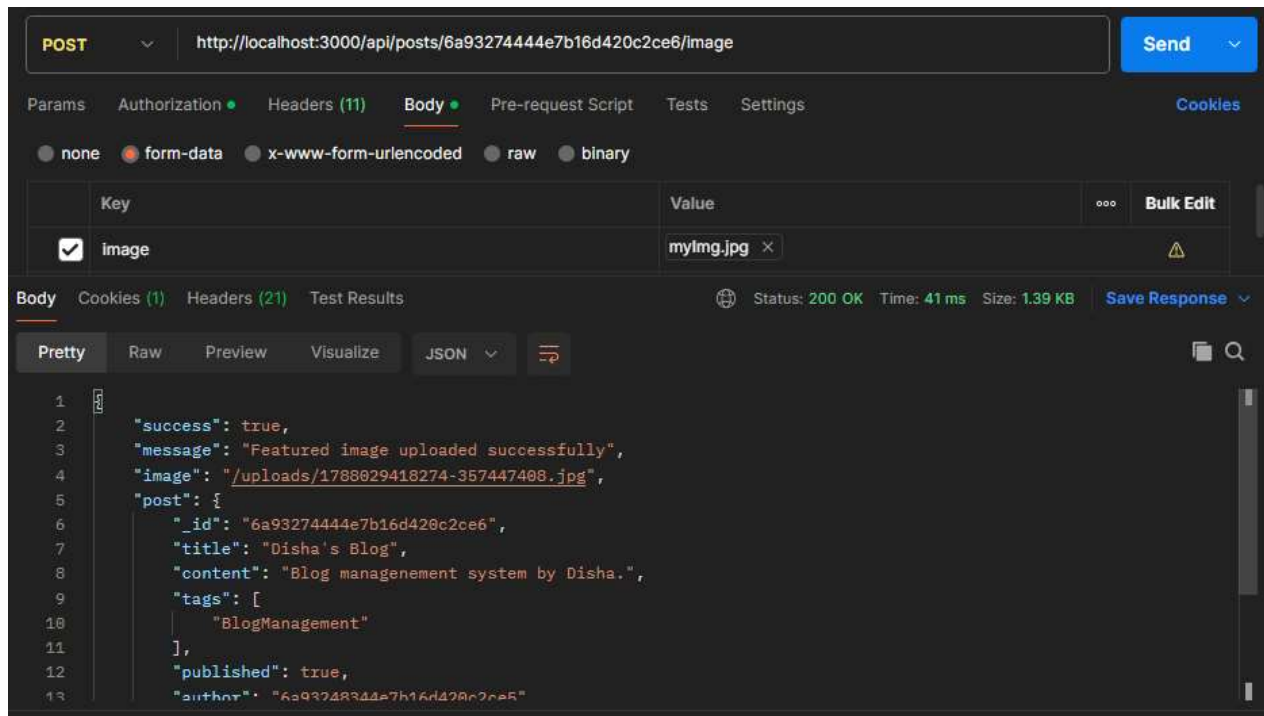
Blog managenement system by Disha.

Published: Yes

Tags: BlogManagement

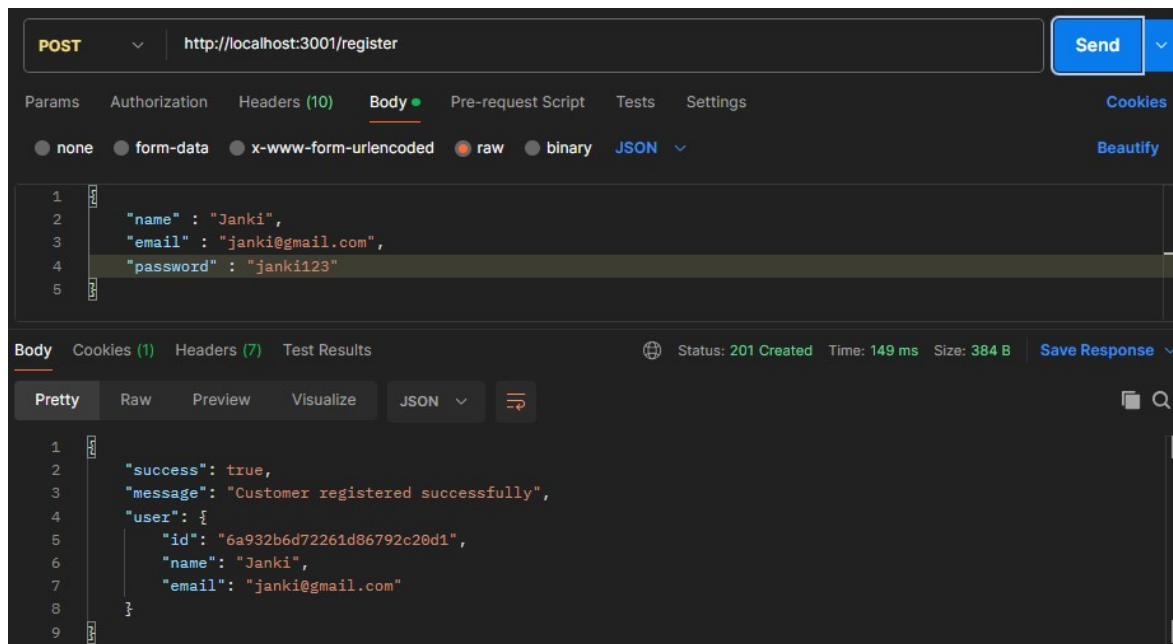
[Edit](#) [Delete](#)

- File Uploading



Que.2: Secure Signup for Bookstore Customers

Output :

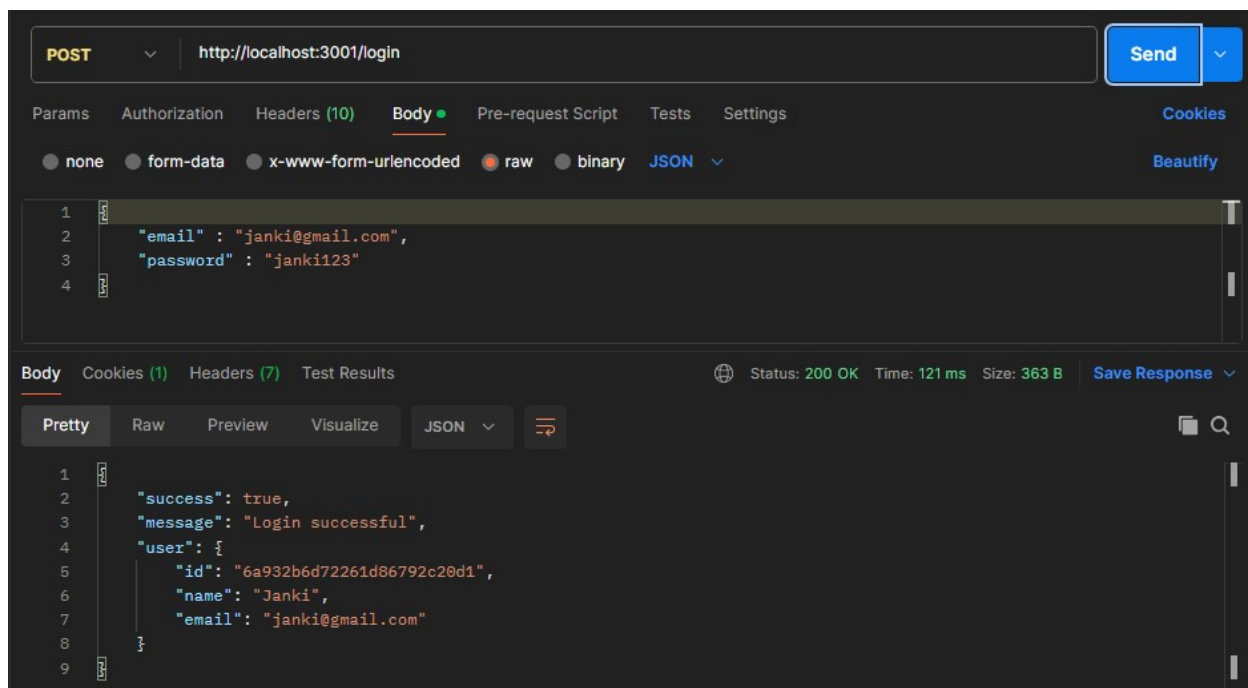


The screenshot shows a REST client interface with a POST request to `http://localhost:3001/register`. The request body is a JSON object with the following fields:

```
1 {
2   "name": "Janki",
3   "email": "janki@gmail.com",
4   "password": "janki123"
5 }
```

The response is displayed in the "Body" tab, showing a successful status of 201 Created. The response body is a JSON object:

```
1 {
2   "success": true,
3   "message": "Customer registered successfully",
4   "user": {
5     "id": "6a932b6d72261d86792c20d1",
6     "name": "Janki",
7     "email": "janki@gmail.com"
8   }
9 }
```



The screenshot shows a REST client interface with a POST request to `http://localhost:3001/login`. The request body is a JSON object with the following fields:

```
1 {
2   "email": "janki@gmail.com",
3   "password": "janki123"
4 }
```

The response is displayed in the "Body" tab, showing a successful status of 200 OK. The response body is a JSON object:

```
1 {
2   "success": true,
3   "message": "Login successful",
4   "user": {
5     "id": "6a932b6d72261d86792c20d1",
6     "name": "Janki",
7     "email": "janki@gmail.com"
8   }
9 }
```


Que.3: Issuing a Digital Membership Token on Login

Output :

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3002/api/login
- Body:**

```
{  "email": "customer@gmail.com",  "password": "password123"}
```
- Status:** 200 OK
- Time:** 17 ms
- Size:** 514 B
- Response Body (JSON):**

```
{  "success": true,  "message": "Login successful",  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjdXN0b21lcjEyMyIsImVtYWlsIjoieyY3VzdG9tZXJAZ21haWwuy29tIiwicm9sZSI6ImN1c3RvbWVYIiwiaWF0IjoxNzg4MDMwMDc1LCJleHAiOjE3ODQwMzNzV9.OGSH-E1Ag0310J96qAhp9KKDAaS5WE1X47XWo5Bv_VE"}
```

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3002/api/orders
- Authorization:** Bearer Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjdXN0b21lcjEyMyIsImVtYWlsIjoieyY3VzdG9tZXJAZ21haWwuy29tIiwicm9sZSI6ImN1c3RvbWVYIiwiaWF0IjoxNzg4MDMwMDc1LCJleHAiOjE3ODQwMzNzV9.OGSH-E1Ag0310J96qAhp9KKDAaS5WE1X47XWo5Bv_VE
- Status:** 200 OK
- Time:** 10 ms
- Size:** 482 B
- Response Body (JSON):**

```
{  "success": true,  "message": "Order history accessed successfully",  "user": {    "userId": "customer123",    "email": "customer@gmail.com",    "role": "customer",    "iat": 1788030075,    "exp": 1788033675  },  "orders": []}
```

Que.4: Defining Customer and Admin Roles

Output :

```
JS User.js X
Question4_Roles > models > JS User.js > ...
1  const mongoose = require("mongoose");
2  ⚡
3  const userSchema = new mongoose.Schema({
4      name: {
5          type: String,
6          required: true
7      },
8
9      email: {
10         type: String,
11         required: true,
12         unique: true
13     },
14
15     role: {
16         type: String,
17         enum: ["customer", "admin"],
18         default: "customer"
19     }
20 });
21
22 module.exports = mongoose.model("User", userSchema);
```


JS app.js X

Question4_Roles > JS app.js > then() callback > user > email

```
2 const User = require("../models/User");
3 mongoose.connect("mongodb://127.0.0.1:27017/bookstoreDB")
4   .then(async () => {
5     console.log("MongoDB connected successfully");
6     const user = await User.create({
7       name: "Disha",
8       email: "disha@gmail.com"
9     });
10
11     console.log("User created successfully:");
12     console.log(user);
13
14     console.log("Role:", user.role);
15
16     mongoose.connection.close();
17   })
18   .catch((error) => {
19     console.log("Error:", error.message);
20   });
```

Problems Output Debug Console Terminal Ports

```
les> node app.js
MongoDB connected successfully
User created successfully:
{
  name: 'Disha',
  email: 'disha@gmail.com',
  role: 'customer',
  _id: new ObjectId('6a932e7877ad81454d0915e3'),
  __v: 0
}
```

Que.5: Admin-Only Access to the Inventory Dashboard

Output :

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3004/api/admin/inventory
- Authorization:** Bearer Token (eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
- Status:** 200 OK
- Time:** 22 ms
- Size:** 398 B
- Response Body (JSON):**

```
1 {
2   "success": true,
3   "message": "Inventory data accessed successfully",
4   "inventory": [
5     {
6       "book": "Atrapi",
7       "stock": 70,
8       "sales": 57
9     },
10    {
11      "book": "Satyabhama",
12      "stock": 10,
13      "sales": 8
14    }
15  ]
}
```

Que.6: Cancelling an Order – Customer or Admin Only

Output :

DELETE

Params **Authorization** Headers (9) Body Pre-request Script Tests Settings Cookies

Type Bearer T... Token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 22 ms Size: 385 B

Pretty Raw Preview Visualize JSON

```
1  {
2    "success": true,
3    "message": "Order cancelled successfully",
4    "order": {
5      "id": "order1",
6      "customerId": "customer123",
7      "book": "Satyabhama",
8      "status": "cancelled"
9    }
10 }
```

DELETE

Params **Authorization** Headers (9) Body Pre-request Script Tests Settings Cookies

Type Bearer T... Token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

Body Cookies (1) Headers (7) Test Results Status: 403 Forbidden Time: 6 ms Size: 319 B

Pretty Raw Preview Visualize JSON

```
1  {
2    "success": false,
3    "message": "Forbidden. You can cancel only your own orders."
4  }
```

--- End of the file ---